

Лекция № 2

Операции и выражения

1 Операция присвоения

Пример:

```
e = 4;
```

```
a = b = c = e + 2;
```

Явное приведение типа:

объект = (тип)выражение;

Пример:

```
float f = 5.7;
```

```
int c1 = f;
```

```
int c2 = (int) f;
```

Комбинированная операция присвоения:

a **оп** **=** **b** $\Rightarrow$ **a** **=** **a** **оп** **b** $\Rightarrow$

```
int x = 2;
```

```
x += 3; // x=5
```

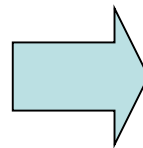
```
x *= 2; // x = ???
```

2 Унарные операции

&	получение адреса операнда
*	обращение по адресу (разыменование)
-	унарный минус, меняет знак арифметического операнда
~	побитовое инвертирование внутреннего двоичного кода (побитовое отрицание)
!	логическое отрицание (НЕ). В качестве логических значений используется 0 - ложь и не 0 - истина, отрицанием 0 будет 1, отрицанием любого ненулевого числа будет 0.
++	увеличение на единицу: префиксная операция - увеличивает операнд до его использования, постфиксная операция увеличивает операнд после его использования.
--	уменьшение на единицу: префиксная операция - уменьшает операнд до его использования, постфиксная операция уменьшает операнд после его использования.
sizeof	вычисление размера (в байтах) для объекта того типа, который имеет операнд

```
#include <stdio.h>
```

```
int main()  
{  
    int a = 10;  
    int *adr;  
    adr = &a;  
    printf("adr = %p", adr);  
    printf("\n");  
    printf("*adr = %d", *adr);  
}
```



```
adr = 0x7ffd826c6ad4  
*adr = 10
```

```
#include <stdio.h>
```

```
int main()  
{
```

```
    int value = 2019;  
    char ch = 'a';
```

```
    printf("V = %d\n", value);  
    printf("V = %d\n", ++value);  
    printf("V = %d\n", value++);  
    printf("V = %d\n", value);  
    printf("V = %d\n", --value);  
    printf("V = %d\n", value--);  
    printf("V = %d\n", value);
```

```
    printf("!value = %d\n", !value);  
    printf("value = %d\n", value);  
    printf("Size1 = %d\n", sizeof(ch));  
    printf("Size2 = %d\n", sizeof(char));  
    printf("Size3 = %d\n", sizeof(value));  
    return 0;
```

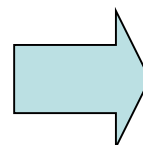
```
}
```

Префиксная форма операций:

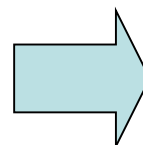
++ операнд -- операнд

Постфиксная форма:

операнд ++ операнд --.



```
V = 2019  
V = 2020  
V = 2020  
V = 2021  
V = 2020  
V = 2020  
V = 2019
```



```
!value = 0  
value = 2019  
Size1 = 1  
Size2 = 1  
Size3 = 4
```

3 Бинарные операции

Аддитивные операции

+	бинарный плюс (сложение арифметических операндов)
-	бинарный минус (вычитание арифметических операндов)

Мультипликативные операции

*	умножение операндов арифметического типа
/	деление операндов арифметического типа (если операнды целочисленные, то выполняется целочисленное деление)
%	получение остатка от деления целочисленных операндов

Примеры:

Int x = 10;

float y1 = x / 3;

float y2 = (float) x / 3;

float y3 = x / 3.;

int y4 = 5.2 / 2;

Int y5 = x % 3;

Операции сдвига

<<	сдвиг влево битового представления значения левого целочисленного операнда на количество разрядов, равное значению правого операнда
>>	сдвиг вправо битового представления значения правого целочисленного операнда на количество разрядов, равное значению правого операнда

Пример:

```
char n = 11; // n=00001011  
n = n << 3; // n=01101000
```

```
char b = 0x06; // 0x06 = 00000110  
b = b << 6;    // b=10000000
```

$$X \ll N = X \cdot 2^N$$

$$X \gg N = X / 2^N$$

Поразрядные операции

&	поразрядная конъюнкция (И) битовых представлений значений целочисленных операндов
	поразрядная дизъюнкция (ИЛИ) битовых представлений значений целочисленных операндов
^	поразрядное исключающее ИЛИ битовых представлений значений целочисленных операндов

Операции сравнения

<	меньше, чем
>	больше, чем
<=	меньше или равно
>=	больше или равно
==	равно
!=	не равно

Логические бинарные операции

&&	конъюнкция (И) целочисленных операндов или отношений, целочисленный результат ложь (0) или истина (1)
	дизъюнкция (ИЛИ) целочисленных операндов или отношений, целочисленный результат ложь (0) или истина (1)

Приоритет выполнения операций?

Примеры

```
int a = 2, b = 5, c;
```

```
c = ++b * ++a;
```

```
c = a*b>b++;
```

```
c = (a | b) || (a & b);
```

```
c = a >> b--;
```

```
c = -a + ++b;
```

```
c = a+++b;
```

```
c = (b==5);
```

```
c = ~a && b + 8;
```

```
c = (~a && b) + 8;
```

```
c = !(a && b) + 8;
```

4 Тернарная операция

операнд1 ? операнд2 : операнд3

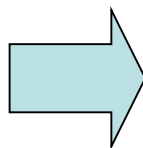
```
#include <stdio.h>

int main()
{
    int x = 3;

    x < 0 ? --x : x;
    printf("x = %d\n", x);

    x == 3 ? x = 10 : x;
    printf("x = %d\n", x);

    return 0;
}
```



```
x = 3
x = 10
```